OpenCell Design: An Open-Source Python Library for Construction, Energetic and Technoeconomic Modelling of Battery Cell Designs

Nicholas Siemons*[1] and Adrian Yao[1]

[1]Department of Materials Science and Engineering, Stanford University, Stanford, CA 94305, USA

March 17, 2026

Abstract

The design and optimization of metal-ion battery cells requires the integration of material properties and cell architecture into a coherent modeling framework. We present OpenCell Design, an open-source Python library that provides a hierarchical, composable API for building virtual battery cells from individual components and materials. The software implements a physically intuitive modeling hierarchy spanning materials, formulations, electrodes, layups, electrode assemblies, encapsulations and complete cells. Users can construct cylindrical, prismatic, and pouch cell architectures using round or flat wound jelly rolls, stacked assemblies, or z-fold stacked assemblies. Four key features enable practical design workflows using OpenCell Design. First, a hierarchical update propagation system allows users to modify any parameter at any level of the cell hierarchy, with changes automatically propagating upward to update all dependent calculations. Second, a modular object-oriented architecture allows users to extend the library with custom components such as new materials, current collector geometries, and entirely new cell formats by subclassing the provided base classes. Third, a compact binary serialization format allows cell designs to be saved, shared, and version controlled. Fourth, interactive browser-based visualizations provide immediate visual feedback through cross-section views, capacity curves, and hierarchical cost and mass breakdowns.

Keywords: battery, cell design, open-source, Python, technoeconomic modeling

*Corresponding author

1 Introduction

The global transition to electrified transportation and grid-scale energy storage has placed increasing demands on energy storage technologies [1, 2]. Metal-ion batteries have emerged as the dominant technology for applications ranging from portable electronics to electric vehicles and stationary storage [3, 4]. In such applications, thermodynamic properties—such as specific and volumetric energy density—are critically important to their performance and to the progress of such technologies. Equally important are technoeconomic metrics—such as cost per unit energy stored—as these properties ultimately drive the adoption of energy storage technologies [5, 6]. Understanding the design and material drivers behind cell performance and cost is critical for further research, development and investment in metal-ion batteries.

Evaluating the drivers of performance and cost requires modeling of the cell at multiple scales. A single cell comprises dozens of interacting parameters and many individual components, spanning individual materials, electrode formulations, current collectors, separators, electrodes and encapsulations. Changes at the material level propagate through to cell-level performance and cost in complex ways, as do design choices associated with certain cell components. Furthermore, changes to the cell design may equally manifest at the cell level in important ways.

OpenCell Design is an open-source Python library that addresses this challenge by providing a hierarchical, composable API for building virtual battery cells from their individual components and materials. OpenCell Design calculates mass and cost at every level of the hierarchy, enabling direct assessment of economic viability alongside kinetic performance parameters.

2 Existing Software

Several software tools have been developed to support battery cell modeling, each addressing different aspects of the design challenge. PyBaMM [7] is an open-source library for physics-based electrochemical simulation of batteries. It provides a flexible framework for constructing and solving models such as the Doyle–Fuller–Newman equations, enabling detailed analysis of ion transport, reaction kinetics, and thermal behavior during charge and discharge. PyBaMM excels at predicting dynamic cell behavior and has become widely adopted in the research community. However, its focus is on electrochemical phenomena rather than cell engineering: it does not model the geometric construction of cells from discrete components, nor does it calculate mass and cost breakdowns from the material level upward.

BatPaC [8] and CAMS [9], developed by Argonne National Laboratory and the Faraday Institution respectively, take a complementary approach. They provide models that estimate the manufacturing cost and performance of battery packs and cells as a function of the cell design. However, both BatPaC and CAMS are implemented as Microsoft Excel workbooks, which limits extensibility and integration with programmatic workflows. A further limitation of these approaches is that they are unidirectional models—the outputs rely on a single set of parameter inputs, and bi-directional parameter setting is not possible.

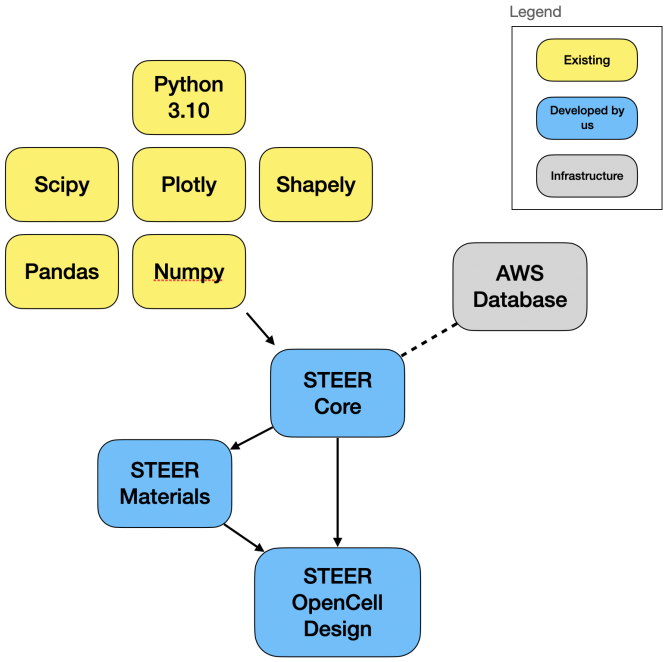


Figure 1: Software dependencies and architecture of OpenCell Design.

Other tools exist for specific aspects of battery design, including electrode formulation calculators and capacity matching spreadsheets, but these typically address isolated aspects of cell design rather than the complete hierarchy from materials to cells. A gap remains for an open-source, programmatic and API-friendly tool that combines hierarchical cell construction with integrated cost and performance calculations in an extensible, composable framework.

3 Overview of OpenCell Design

OpenCell Design is built around a hierarchical modeling architecture that mirrors the physical construction of a cell (Figure 1). It is written in Python, using standard and highly optimized calculation and visualization libraries (Figure 2) [10, 11, 12] for mathematical logic and plotting. OpenCell Design is connected to a database containing standard materials and previously configured cell designs. Additionally, it relies on two infrastructure packages, `steer_core` and `steer_materials`. Both are open source and available under the same licensing as OpenCell Design.

OpenCell Design follows these key design principles:

- Components are realized through Python classes, enabling a fully object-oriented model.
- Components inherit from base classes (Figure 3 for an example), allowing common logic to exist in the base class and thus allowing new components to be easily defined. Base classes are also implemented as 'abstract base classes', enforcing functionality of child classes where needed.

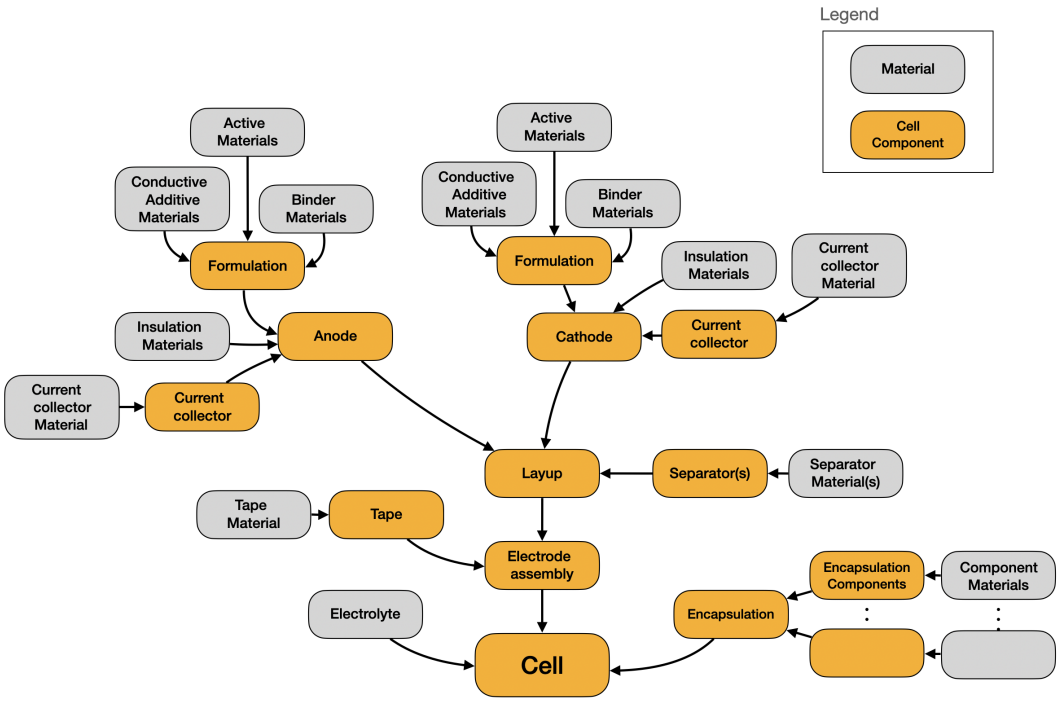


Figure 2: Hierarchical modeling architecture of OpenCell Design, illustrating the composable structure from materials through to complete cells.

- Component classes only contain logic which is relevant to that component. Logic associated with serialization, plotting etc. are contained in mixins in `steer_core`.
- Component attributes are private (with a preceding underscore) and are stored in SI units. They are accessible as 'properties', which perform rounding and conversion to user-friendly units.
- Attributes are explicitly set using 'setters', which perform conversion to SI units, input validation, and carry out logic to keep the model physics-constrained.
- Component properties are calculated in the class furthest down the hierarchy tree possible. For example, specific capacity is defined in the active material and formulation components. Areal capacities are defined in the electrode after the formulation has been coated onto the current collector. Capacity is calculated at the cell level, after potentially multiple electrode assemblies may come together.

The property/setter approach allows for OpenCell Design to be an extensible, bidirectional model. For example, users can modify the n/p ratio of a cell by manipulating the mass loading of the cathode and anode. They can likewise directly set the n/p ratio of the layup, which modifies the cathode and anode mass loading according to a control mode to achieve the desired n/p ratio.

The Pythonic implementation gives access to the many powerful and highly optimized Python libraries for carrying out numerical calculations. Examples include the calculation of

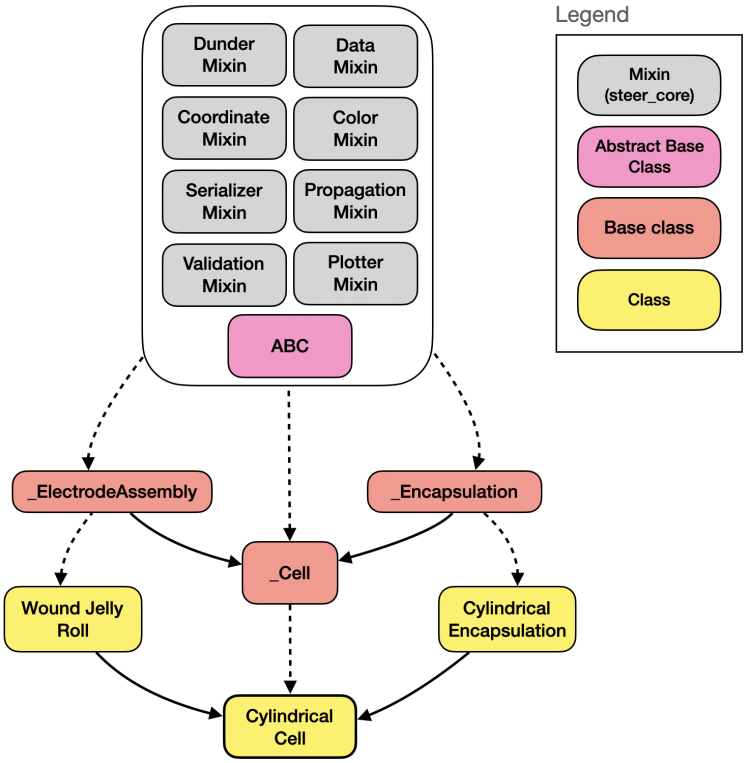


Figure 3: Example base class structure demonstrating the inheritance hierarchy used in OpenCell Design.

thickness-dependent jelly roll spirals via an adaptive RK4 scheme, optimized with a just-in-time compiler [13], and numerical minima search engines such as the BrentQ algorithm [14], implemented in SciPy [11].

The object-oriented approach allows for easy extensibility with novel cell types. To demonstrate this, in OpenCell Design we include a class definition for a novel flex-frame cell, representing one of the cell architectures being used to realize solid-state cell technologies. We can fully define the encapsulation in approximately 700 lines of code, and the cell in approximately 300 lines of code, after which the full cell can be modeled through inheriting the cell base classes, and using composition with existing cell components.

4 Constructing a Cell

Cell construction follows the hierarchical structure shown in Figure 1. At the materials level, users define active materials, binders, conductive additives, separator materials, current collector materials, and encapsulation materials. Each material is parameterized by density and specific cost. Active materials additionally carry first-cycle half-cell voltage–capacity curves. Alternatively, users can access standard materials using the `.from_database()` API for any material class.

Active materials, binders and conductive additives are combined into electrode formula-

tions. Any number of active materials, binders or conductive additives can be added to the formulation. Formulations are then paired with current collectors to create electrodes, with the option to also add electrical insulation strips.

The cathode and anode electrodes are arranged with separators into a layup. For wound cells, a `Laminate` places the cathode and anode between top and bottom separators; for stacked cells, a `MonoLayer` uses a single separator between electrode pairs.

The layup is wound or stacked into an electrode assembly. For cylindrical cells, a `WoundJellyRoll` winds the laminate around a cylindrical mandrel, calculating the number of turns and final outer radius from the layup thickness and mandrel diameter. For prismatic cells, a `FlatWoundJellyRoll` produces a racetrack-shaped winding with flat sides. For pouch and prismatic cells using stacked configurations, `ZFoldStack` and `PunchedStack` assemblies calculate the number of layers required to fill the available volume. The assembly aggregates the total active area, capacity, and mass from the repeated layup units, and adds termination tape for wound designs. Finally, the electrode assembly is combined with an encapsulation and electrolyte to produce the complete cell.

Example scripts on building cells of different formats and internal constructions are included in the supporting information.

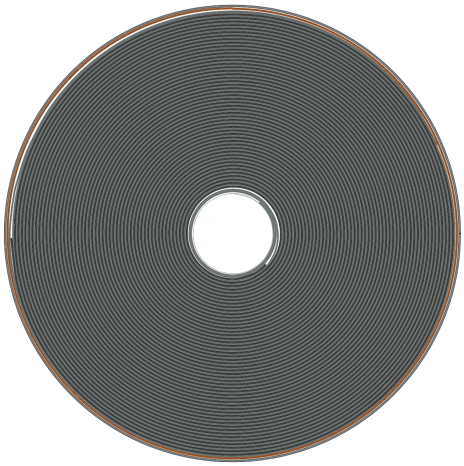
5 Modifying a Cell

A key challenge in hierarchical cell modeling is maintaining consistency when parameters change. Modifying a property deep in the hierarchy—such as the density of an active material—potentially affects calculations at every level above it: the formulation's blended density, the electrode's coating thickness, the layup's capacity balance, and ultimately the cell's energy and cost. Manually propagating these changes by reassigning objects at each level is tedious and error-prone.

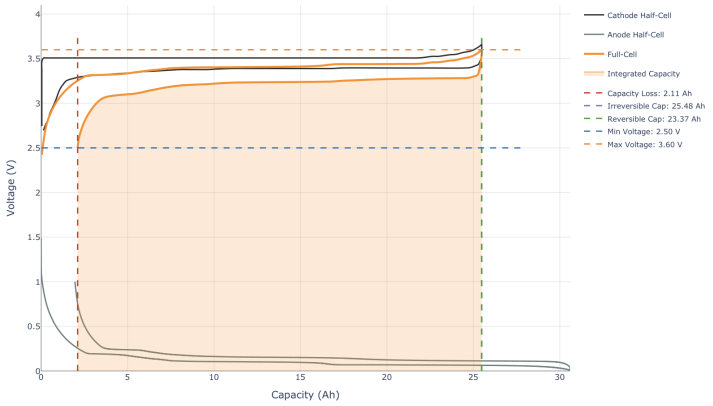
OpenCell Design addresses this through a propagation system that automatically updates dependent calculations when parameters change. Each object in the hierarchy maintains a reference to its parent and knows which attribute it occupies on that parent. When a user modifies a property and calls `object.propagate_changes()`, the system triggers the parent's setter by re-assigning the modified object. This invokes the parent's property recalculation logic, which updates all derived quantities at that level. The process then continues upward through the hierarchy until reaching the cell.

For cases requiring finer control, the `update()` method propagates changes by a single level rather than all the way to the cell. This allows users to modify parameters at intermediate levels between propagation steps, enabling complex design operations that depend on the order of calculations. This level-by-level approach enables studies that would otherwise require rebuilding the cell from scratch, such as comparing designs at constant volume, constant capacity, or constant N/P ratio while varying other parameters.

Examples of using both `object.propagate_changes()` and `object.update()` are included in the supporting information.



(a) Cross-section visualization of a cylindrical cell showing the internal arrangement of electrodes, separators, and current collectors.



(b) Voltage–capacity curves showing charge and discharge behavior with annotated capacity metrics.

Figure 4: OpenCell Design visualizations: (a) geometric cross-section view and (b) electrochemical voltage–capacity curves.

6 Visualizations

OpenCell Design provides interactive browser-based visualizations using Plotly [12], enabling immediate visual feedback during design iteration. Visualization methods are available at multiple levels of the hierarchy, allowing users to inspect individual components or the complete cell.

Geometric visualizations can show the physical structure of the cell (Figure 4a). Cross-section views display the internal arrangement of electrodes, separators, and current collectors within wound or stacked assemblies—for cylindrical cells, this reveals the spiral structure of the jelly roll; for stacked cells, it shows the layered electrode arrangement. Top-down views display the cell from above, showing tab positions, electrode dimensions, and encapsulation geometry. Side views are available for prismatic and pouch cells to show the stacking arrangement and seal regions.

Electrochemical visualizations display voltage–capacity behavior (Figure 4b). Capacity plots show both charge and discharge curves, with annotations marking the operating voltage window, reversible capacity, irreversible capacity, and capacity loss. These plots update automatically when cell parameters change, providing immediate feedback on how design choices affect electrochemical performance.

Technoeconomic visualizations break down mass and cost contributions using interactive sunburst charts. These hierarchical diagrams show how each component contributes to total cell mass and cost, from raw materials at the innermost ring through to the complete cell at the outermost ring. Users can click to zoom into specific branches of the hierarchy, making it straightforward to identify the dominant cost or mass drivers in a design.

All visualization methods return Plotly figure objects that can be displayed in Jupyter

notebooks, saved to image files, or embedded in web applications.

7 Quality Control

OpenCell Design includes a comprehensive test suite to ensure correctness and prevent regressions during development. The test suite comprises 12 test modules containing over 700 individual test cases, organized to mirror the package structure: separate modules test materials, formulations, electrodes, current collectors, separators, layups, electrode assemblies, containers, and complete cells.

8 Conclusion

OpenCell Design provides an open-source, programmatic framework for designing and analyzing metal-ion battery cells. By implementing a hierarchical modeling architecture that mirrors the physical construction of cells—from raw materials through to complete cell assemblies—OpenCell Design enables users to explore the relationships between material properties, design choices, and cell-level performance and cost metrics.

The propagation system allows rapid design iteration by automatically updating derived quantities when parameters change, eliminating the need to manually rebuild cell models. The modular, object-oriented architecture supports extension with custom materials, current collector geometries, and entirely new cell formats through subclassing. Serialization capabilities enable cell designs to be saved, shared, and version-controlled, while interactive visualizations provide immediate feedback on cell geometry, electrochemical behavior, and cost breakdowns.

The library addresses a gap between electrochemical simulation tools, which focus on physics-based modeling of cell behavior, and spreadsheet-based cost models, which lack extensibility and programmatic access. By providing both capabilities in a composable Python API, OpenCell Design supports workflows ranging from early-stage design exploration to systematic optimization studies. The software is freely available under the GNU Affero General Public License v3.0, with documentation and examples provided in the repository.

References

[1] Steven Chu and Arun Majumdar. Opportunities and challenges for a sustainable energy future. *Nature*, 488(7411):294–303, 2012.

[2] Bruce Dunn, Haresh Kamath, and Jean-Marie Tarascon. Electrical energy storage for the grid: A battery of choices. *Science*, 334(6058):928–935, 2011.

[3] John B Goodenough and Kyu-Sung Park. The Li-ion rechargeable battery: A perspective. *Journal of the American Chemical Society*, 135(4):1167–1176, 2013.

[4] George E Blomgren. The development and future of lithium ion batteries. *Journal of The Electrochemical Society*, 164(1):A5019–A5025, 2017.

[5] Björn Nykvist and Måns Nilsson. Rapidly falling costs of battery packs for electric vehicles. *Nature Climate Change*, 5(4):329–332, 2015.

[6] Micah S Ziegler and Jessika E Trancik. Re-examining rates of lithium-ion battery technology improvement and cost decline. *Energy & Environmental Science*, 14(4): 1635–1651, 2021.

[7] Valentin Sulzer, Scott G Marquis, Robert Timms, Martin Robinson, and S Jon Chapman. Python Battery Mathematical Modelling (PyBaMM). *Journal of the Open Research Software*, 9(1):14, 2021.

[8] Paul A Nelson, Shabbir Ahmed, Kevin G Gallagher, and Dennis W Dees. BatPaC: Battery Manufacturing Cost Estimation. Technical report, Argonne National Laboratory, 2019.

[9] Faraday Institution. CAMS: Cost and Manufacturing Simulation. Technical report, The Faraday Institution, 2023.

[10] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[11] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[12] Plotly Technologies Inc. Plotly: Collaborative data science. `https://plot.ly`, 2015.

[13] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.

[14] Richard P Brent. An algorithm with guaranteed convergence for finding a zero of a function. *The Computer Journal*, 14(4):422–425, 1971.